

# Advanced Operating Systems

20MCA172

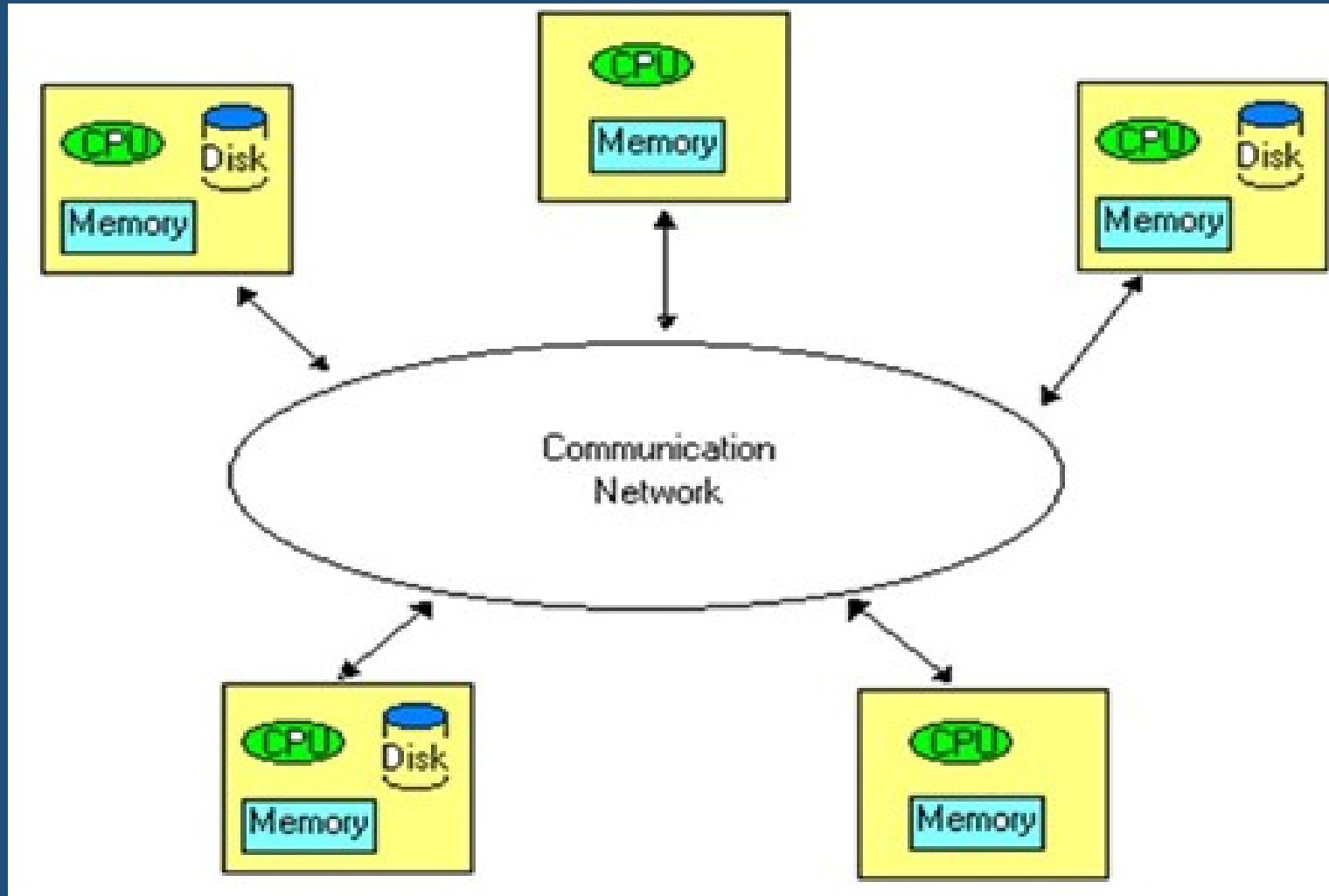
(Elective )

# Distributed Operating Systems

# Distributed Operating Systems

- The term distributed system is used to describe a system with the following characteristics:
  - It consists of several computers that do not share a memory or a clock.
  - The computers communicate with each other by exchanging messages over a communication network.
  - Each computers has its own memory and runs its own operating system.
  - The resource owned and controlled by a computer are said to be local to it.

# Architecture of a Distributed Systems



# Advantages of Distributed Systems

- **Resource Sharing**
  - A computer can request a service from another computer by sending a request to it over the communication network.
  - Hardware and software resources can be shared among computers.
  - Eg: a printer, a compiler, a text processor, a database at a computer can be shared with remote computers.
- **Enhanced Performance**
  - Capable for providing rapid response time and higher system throughput.
  - Many task can be concurrently executed at different computers.
  - Can employ a load distributing technique to improve response time.

# Advantages of Distributed Systems

- **Improved Reliability and Availability**
  - Few components of the system can fail without affecting the availability of the rest of the system.
  - Through the replication of data (files and directories) and services, distributed systems can be made fault tolerant.
- **Modular Expandability**
  - New software and hardware resources can be easily added without replacing the existing resources.

# Issues in Distributed Operating Systems

# Issues in DOS

- **Some Important issues are:**
  - 1. Unavailability of up-to-date global knowledge**
  - 2. Naming**
  - 3. Scalability**
  - 4. Compatibility**
  - 5. Process synchronization**
  - 6. Resource management**
  - 7. Security**
  - 8. Structuring of the operating system**



# Issues in DOS

## 1. Unavailability of up-to-date global knowledge

- In case of shared memory system, up-to-date state of all the processes and resources are known.
- Due to the unavailability of a global memory and a global clock, it is practically impossible for a computer to collect up-to-date information about the global state of the distributed computing system.
- A computer does not know the current and complete status of the global state.
- Absence of a global clock arises the question of how to order all the events that occur at different times at different computers present in the system.
- Temporal ordering of events is a fundamental concept in the design and development of distributed systems.

# Issues in DOS

## 2. Naming

- Naming are used to refer to objects.
- Objects that can be named in computer systems include computers, printers, services, files and users.
- Eg: Name Service. [Store names and their physical addresses are used for mapping names to their addresses]
- In distributed system, the directories may replicated and stored at different locations.
- Algorithm depend on structure of names.
- Method of naming objects.

## 3. Scalability

- Systems grow with time.
- The techniques used in designing a system should not result in system unavailability or degraded performance when growth occurs.

# Issues in DOS

## 4. Compatibility

- Refers to the notion of interoperability among the resources in a system.
- Three levels of compatibility that exist in distributed systems are:
  - *Binary level* – all processors execute the same binary instruction, even though the processors may differ in performance and in input-output. Distributed system cannot include computers with different architectures.
  - *Execution level* –if the same source code can be compiled and executed properly on any computer in the system
  - *Protocol level* – if all system components support a common set of protocols. Individual computers can run different Operating Systems.

# Issues in DOS

## 5. Process synchronization

- This is difficult because of the unavailability of shared memory.

## 6. Resource management –

- Concerned with making both local and remote resources available to users in an effective manner.
- Users should be able to access remote resources as easily as they can access local resources.
- In other words, the specific location of resources should be hidden from the users.

# Issues in DOS

## 7. Security –

- The security of a system is the responsibility of the operating system.
- Two issues are authentication and authorization
- Authentication is the process of guaranteeing that an entity is what it claims to be.
- Authorization is the process of deciding what privileges an entity has and making only these privileges.

## 8. Structuring –

- Defines how various parts of the operating system are organized.

# Communication networks

# Communication networks

- All the computers in a distributed system are interconnected through a communication network.
- A computer can exchange messages with other computers and access data stored at another through this network.
- Communication networks are classified into
  1. Wide Area Networks
  2. Local Area Networks

# 1. Wide Area Networks (WANs)

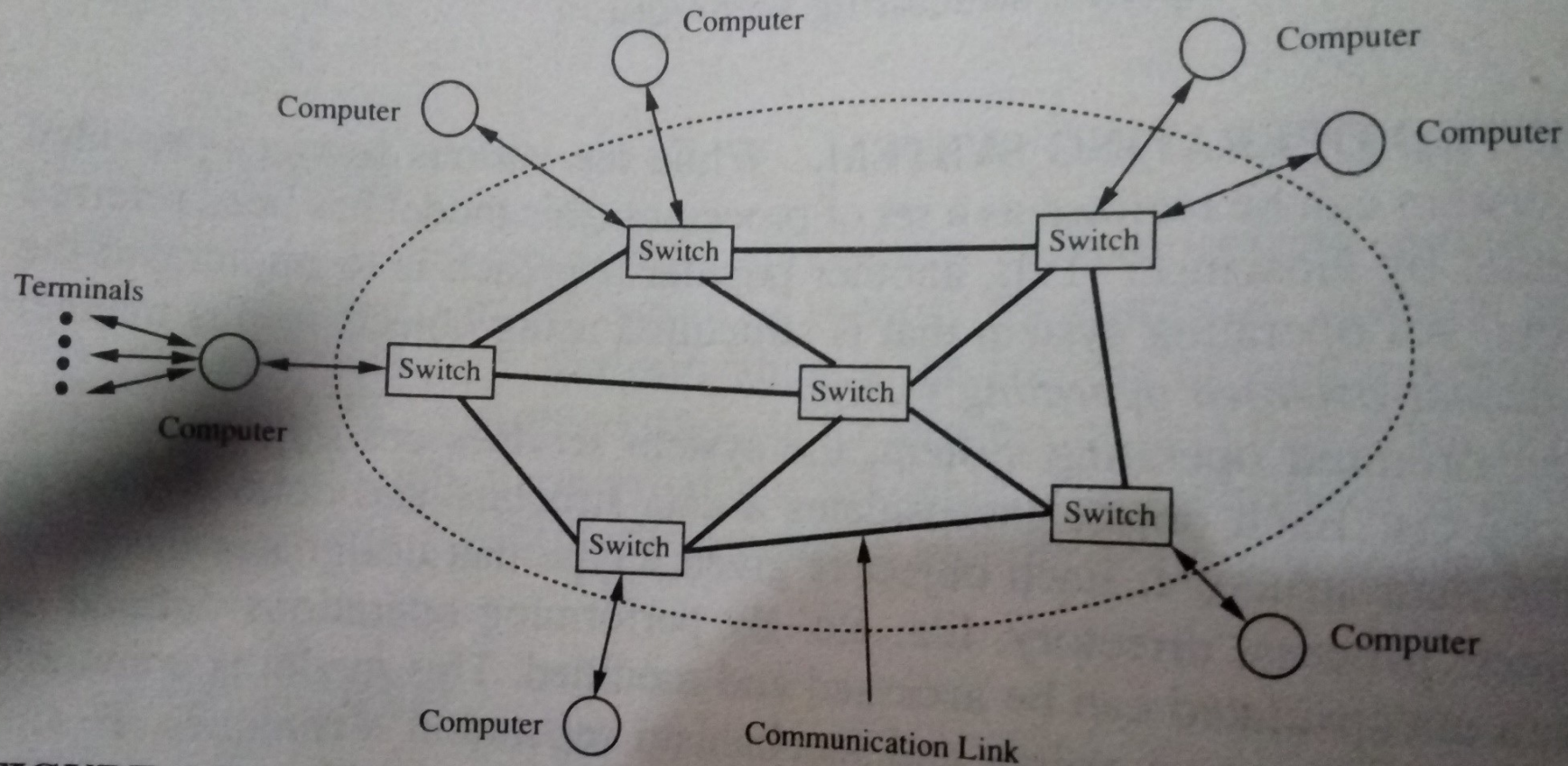
- Used to **interconnect various devices (such as computers and terminals) spread over a wide geographic area that may cover different cities, states and countries.**
- Also referred to as **Long Haul Networks.**
- The communication facility in a WAN consists of switches that are interconnected by communication links, which are established through telephone lines, satellites, microwave links, or any combination of the three.
- WANs employ a technique known as **point-to-point or store-and-forward** where data is transferred between computers through a series of switches.
- **Switches** are special purpose computers responsible for routing data from one point to another through an appropriate path while avoiding network congestion.
- A path or portion of a path become congested due to heavy data communication through that path or due to limited bandwidth.



# 1. Wide Area Networks (WANs)

- The data being communicated in a WAN can be lost for any of the following reasons:
    - Switch crashes
    - Communication link failure
    - Limited buffer capacity at switches
    - Transmission error
- etc...

# 1. Wide Area Networks (WANs)



**FIGURE 4.2**  
A point-to-point network.

# Utilizing communication networks - modes

- The communication network can be utilized in one of the following two modes:
  1. Circuit Switching
  2. Packet Switching

## 1. Circuit switching

- A dedicated path is established between two devices wishing to communicate, and the path remains intact for the entire duration in which the two devices communicate.
- The path is broken when one side terminates the conversation.
- Example – telephone system

# Utilizing communication networks - modes

## 2. Packet switching

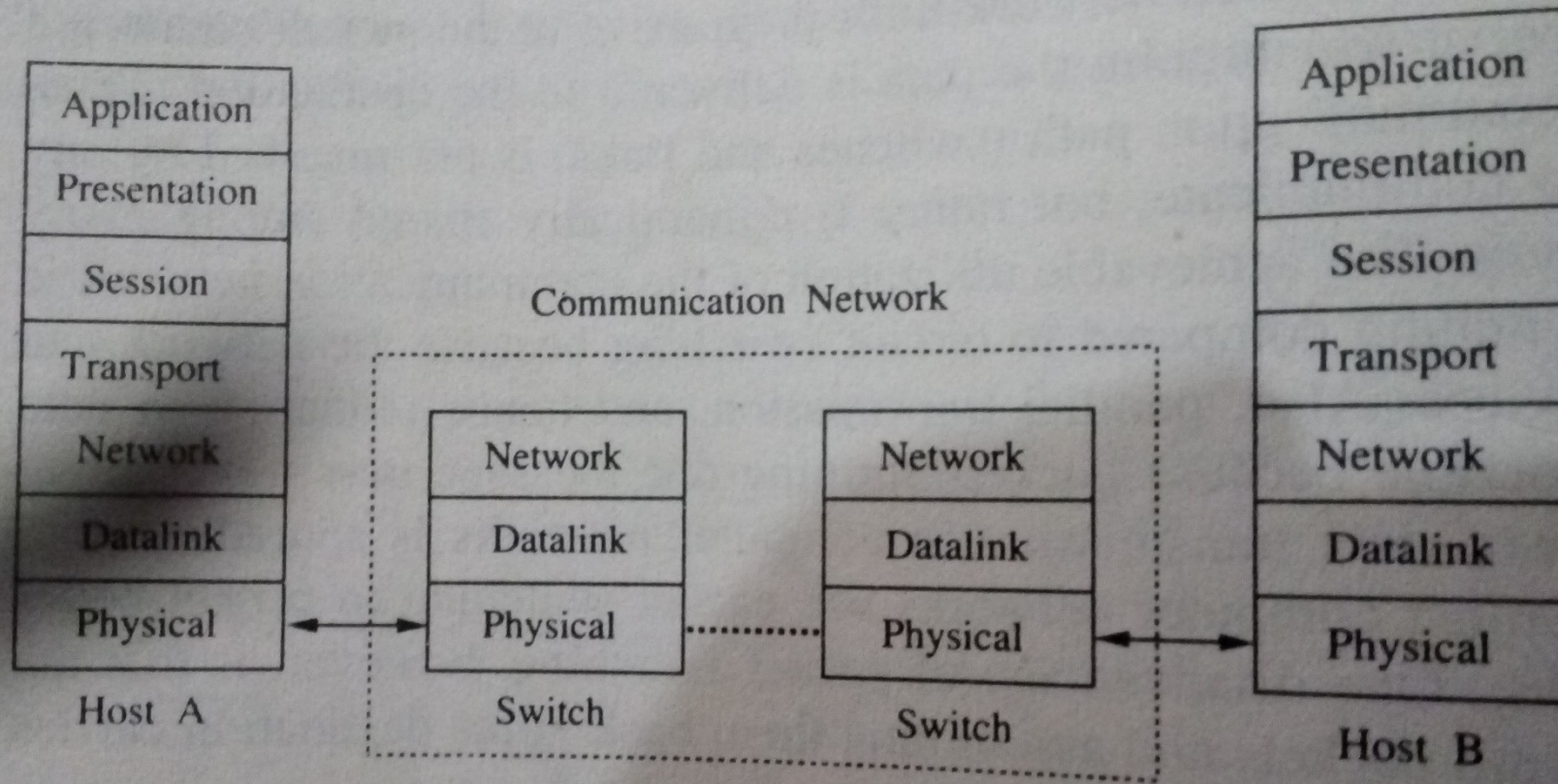
- A connection established between the source device (terminal or computer) and its nearest switch.
- The data or message to be communicated is broken down into smaller units called **packets**, with each packet containing the address of the destination.
- The packets are then sent to the nearest switch.
- These packets are then sent to the nearest switch.
- These packets are routed from one switch to another switch in the communication network until they arrive at the switch connected to the destination device.
- A communication path (switches or links) is not reserved by any two devices wishing to communicate, but rather is dynamically shared among many devices on a demand basis.
- The network can be shared between many devices.

# The ISO OSI Reference Model

- WANs must interconnect heterogeneous types of equipment (Computers, Printers etc).
- These equipment may differ from each other in their speed, word length, information representation or in many other criteria.
- **To communicate in heterogeneous environment, the ISO OSI reference model provides a framework for communication protocols.**
- It organizes the protocols as seven layers and specifies the functions of each layer.
- In this model, user programs run in the application layer.



# ISO OSI reference model



**FIGURE 4.3**

The ISO OSI reference model.

# The ISO OSI Reference Model

- When an application program at computer A wants to send a message to an application program at computer B, the chain of events are:
  - The application layer of A passes the message down to presentation layer.
  - The Presentation layer transforms the data, add a header containing some control information to the message and passes to session layer.
  - The session layer adds its own header to the message and passes to next layer.
  - This continue until the message reaches the physical layer of Computer A.
  - The physical layer on A transmits the raw data bits into physical layer running at computer B. (Message is routed to B through various intermediate switches).
  - At computer A, after receiving message, performs the necessary processing identified by the header and passes on to the next layer.
  - This continues until the message reaches the destination.

# The ISO OSI Reference Model

- The ISO OSI model does not specify how the layers should be implemented.
- Every layer is aware only the protocols and header formats of its own.
- It does not understand the header or the protocols used by other layers.
- Each layer is independent, so any layer can change its protocol without affecting other layers.



## 2. Local Area Networks (LANs)

- A LAN is a communication network that interconnects a variety of data communication devices within a small geographic area.
- **Characteristics of LANs**
  - High data transmission rates.
  - The geographic scope is small, generally confined to a single building or several buildings (eg: college campus)
  - Low transmission error rate.

# Network topologies of LAN

- Widely used network topologies of LAN are:
  1. Bus
  2. Tree
  3. Ring
- The communication media can be coaxial cable, twisted pair wire or optical fibre.

## Bus / Tree topology

- In bus topology, the communication devices transmit data in the form of packets where each packet contains the address of the destination and a message.
- A packet propagates throughout the medium, the bus, and is available to all other devices, but is received by the addressed device.
- In the bus topology, all the devices connected to the LAN are allowed to transmit at any time.
- Since all the devices share a common data path (bus), a protocol to control the access to the bus is necessary.
- A **tree topology LAN** is obtained by interconnecting many bus topology LANs to a common bus.
- Protocols to control access to the bus are,
  1. **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection) **protocol**
  2. **Token bus protocol**

# Bus / Tree topology

## 1. CSMA/CD (Carrier Sense Multiple Access with Collision Detection) **protocol**

- A device wishing to transmit **listens to the medium** to determine whether another transmission is in progress.
- If so, the device **waits** for a random amount of time before trying again.
- If no other transmission is in progress, the device **starts transmitting data** and continues to listen to the medium while it is transmitting.
- If another device starts transmitting simultaneously, the two transmission **collide**.
- If a collision is detected, **a short jamming signal** is transmitted over the bus to inform all the devices that there has been a collision.
- The device will then wait for a random amount of time before attempting to transmit again.
- **Advantage** : Simplicity
- **Disadvantage**: Heavy load, performance degraded due to collision.

# Bus / Tree topology

## 2. Token bus protocol

- Devices physically organized in a bus/tree topology **form a logical ring**, and each device knows the **identity** of the devices preceding and following it.
- Access to the bus is controlled through a **token (Control Packet)**.
- The device **holding the token** is allowed to transmit.
- The device not holding the token can receive message.
- A device is allowed to keep the token for a specific amount of duration, after which it has **to send the token to the device** following it on the **logical ring**.

# Ring topology

- Data is transmitted **point-to-point**.
- At each point, the **address on the packet is copied and checked** to see if the packet is meant for the device connected at that point.
- If the address of the device and the **address in the packet match**, the rest of the packet is copied, otherwise, the entire packet is retransmitted to the next device on the ring.
- **Protocols** to control access to the ring are,
  1. Token ring protocol
  2. Slotted ring protocol

# Ring topology

- Protocols to control access to the ring are,

## 1. Token ring protocol

- Widely used protocol.
- The token is labelled **free** when no device is transmitting.
- When a device wishes to transmit, it waits for the token to arrive, labels the token as busy and retransmit the token. Immediately the device starts transmitting the data.
- The transmitting device will mark the token free when the busy token returns to the device and the device has completed its transmission.
- **Advantage:** Not sensitive to the load on the network.
- **Disadvantage:** Complex. If the token is lost and care must be taken to generate only one token.

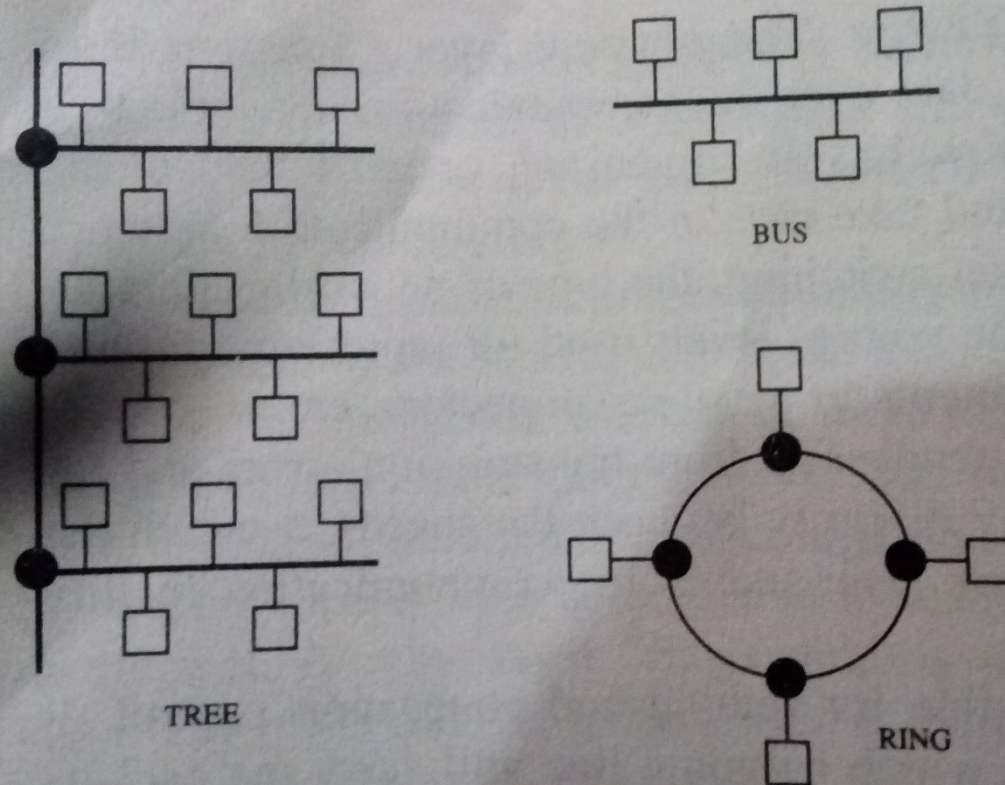
# Ring topology

- Another protocols to control access to the ring are,

## 2. Slotted ring protocol

- A number of **fixed length slots** continuously circulate around the ring.
- A device wishing to transmit data waits for a slot marked *empty* to arrive, marks it *full*, and inserts the destination's address and the data into the slot as it goes by.





**FIGURE 4.4**  
Network topologies.

# Communication primitives

- The communication network provides a means to send and receive raw bit streams of data in distributed systems.
- The communication primitives are **high-level constructs** with which programs use the underlying communication network.
- They influence:
  1. A programmer's choice of algorithms
  2. Ultimate performance of the programs.
  3. Ease of use of the system
  4. Efficiency of applications that are developed for the system
- Communication models that provide communication primitives are,
  1. **The message passing model**
  2. **Remote procedure call**

# The message passing model

- This model provides two basic communication primitives ,
  - **SEND** and **RECEIVE**.
- The **SEND** primitive has two parameters :
  - a **message** and its **destination**.
- The **RECEIVE** primitive has two parameters :
  - the **source of the message** and **a buffer for storing the message**.
- **Example:** client server model
  - The client process needing some service sends a message to the server and waits for a reply message. After performing the task, the server process sends the result in the form of a reply message to the client process.
- The SEND and RECEIVE primitives provide the basic communication ability to programs.
- The *semantics* of these primitives also play a significant role in ease of developing programs.

# The message passing model

- In the standard message passing model, messages are copied three times:
  - From **user buffer** to the **kernel buffer**
  - From **kernel buffer** on the sending computer to the **kernel buffer** on the receiving computer.
  - From the **kernel buffer** on the receiving computer to a **user buffer**.
- This is known as the *buffered* option.
- The two design issues that decide the semantics of these two primitives are:
  1. **BLOCKING VS. NONBLOCKING PRIMITIVES**
  2. **SYNCHRONOUS VS. ASYNCHRONOUS PRIMITIVES**

# 1.BLOCKING VS. NONBLOCKING PRIMITIVES

- With **NONBLOCKING PRIMITIVES**, the SEND primitive **returns control to the user process** as soon as the message is copied from the user buffer onto the kernel buffer.
- The corresponding RECEIVE primitive signals its intention to receive a message and provides a buffer to copy the message. The receiving process may either periodically check for the arrival of a message or be signalled by the kernel upon arrival of a message. Here programming is difficult, but flexible.
- With **BLOCKING PRIMITIVES**, the SEND primitive **does not return control to the user program** until the message has been sent or until an acknowledgement has been received. The user buffer can be reused as soon as the control is returned to the user program. The corresponding RECEIVE primitive does not return control until a message is copied to the user buffer. A reliable RECEIVE primitive automatically sends an acknowledgement, while an unreliable RECEIVE primitive does not send an acknowledgement. Here the behavior of the programs is predictable, and programming is easy., but lack of flexibility in programming and absence of concurrency.

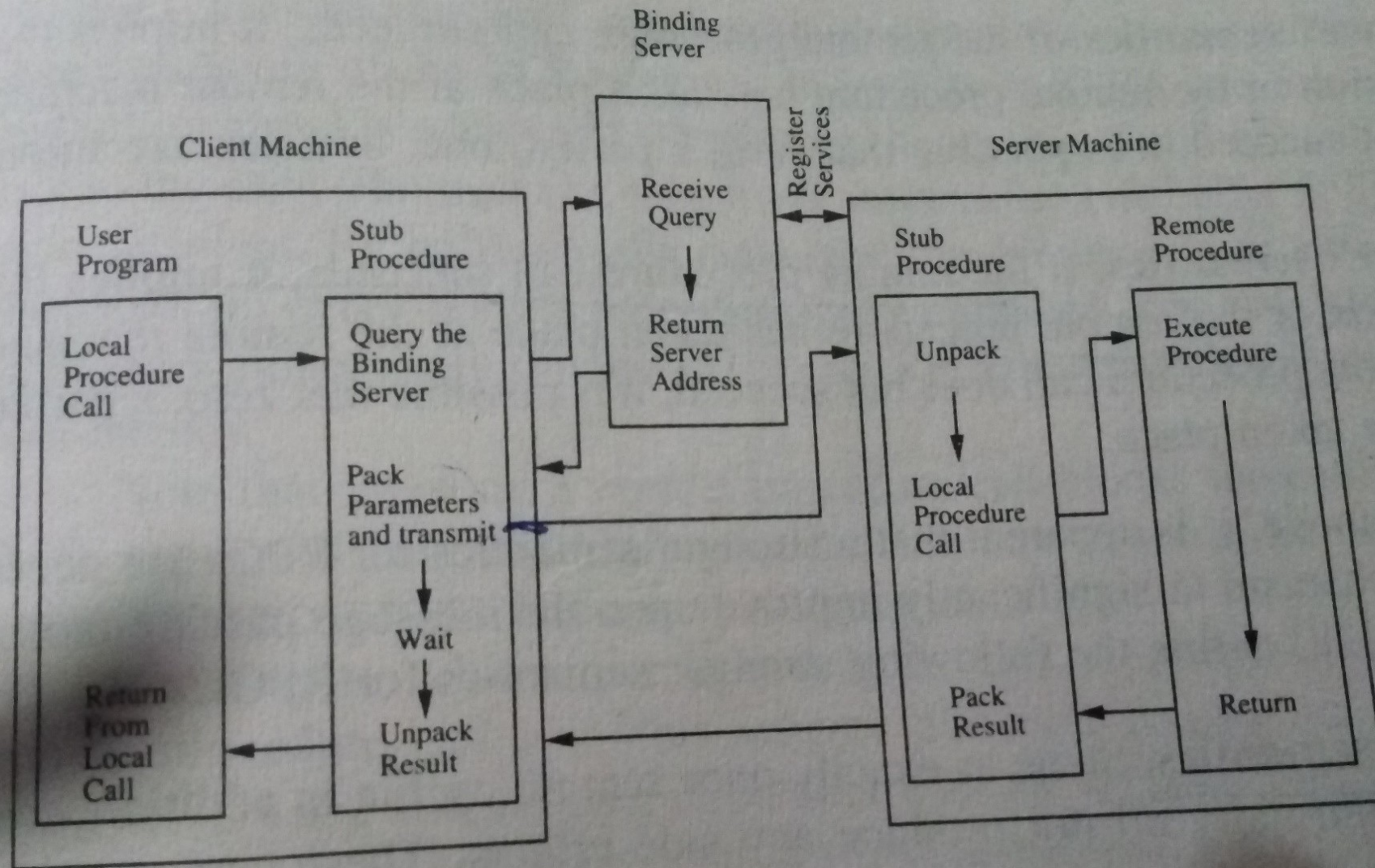
## SYNCHRONOUS VS. ASYNCHRONOUS PRIMITIVES

- With **synchronous primitives**, a **SEND primitive is blocked until a corresponding RECEIVE primitive is executed** at the receiving computer. Also called rendezvous. A blocking synchronous primitive can be extended to an unblocking-synchronous primitive by first copying a message to a buffer at the sending side, and then allowing the process to perform other computational except another SEND.
- With **asynchronous primitives**, the messages are buffered. A **SEND primitive does not block even if there is no corresponding execution of RECEIVE primitive**.
- The corresponding RECEIVE primitive can either be a blocking or a nonblocking primitive. Disadvantage – buffering of messages is more complex and what to do with the messages that are meant for processes that have already died.



# Remote procedure call

- RPC is an interaction between a client and a server, where the client needing a service invokes a procedure at the server.
- On **invoking** a remote procedure, the calling process (the client) is suspended and parameters, if any are passed to the remote machine(the server) where the procedure will execute.
- On completion of the procedure execution, the results are passed back from the server to the client and the client resumes execution as if it had called local procedure.
- While the RPC mechanism looks simple, the issues that arise in designing and implementing are,
  1. Structure
  2. Binding
  3. Parameter and result passing
  4. Error handling, semantics and correctness



**FIGURE 4.5**  
Remote procedure call.



# Design Issues in RPC

- Structure
- Binding
- Parameter and result passing
- Error handling, semantics, and correctness

# Structure

- RPC is based on stub procedures
- Client stub and server stub are used
- Client stub send message to server stub
- Server stub procedure passes result back to client stub procedure

# Binding

## Approach 1

- Binding is a process that determines the remote procedure and the machine on which it will be executed
- Also check the compatibility of parameters passed and the procedure type
- A binding server is used, which store the server machine addresses with the services they provide.
- Servers register with binding server.
- Client stub procedure obtains server address from binding server.

## Approach 2

- Client specifies the machine and service required, binding server returns the port number for communicating with the service required

## Parameter and result passing

- Stub procedure has to convert the parameters and results into an appropriate representation and pack them to a buffer in a form suitable for transmission.
- Conversion is expensive

### Ways to avoid conversion

1. Send parameters along with a code so that receiver can do conversion, this requires the machine to know how to convert all formats. If new representation is there, existing software needs to be updated.
  2. Each data type may have a standard format, sender will convert data to standard format, receiver will convert to local representation.
- Dealing with passing parameters by value and by reference. Passing by value is easy, but by reference is complicated because along with file pointers privileges associated with calling procedure have to be passed.

# Error handling, semantics and correctness

- RPC may fail due to computer failures and communication failures
  1. Messages lost occasionally, result in more than one execution of procedure call.
  2. Client machine crashes, then no machine to receive result. If client machine recovers quickly and reissues RPC, results in more than one execution of procedure call.

## Semantics of RPC

- “At least once” semantics
- “Exactly once” semantics

## Stronger semantics

- “At most once” semantics (Zero-or-one” semantics)

# Lamport's logical clocks

# Lamport's logical clocks

- Two types of Clock in Distributed System,
  - Logical Clock
  - Vector Clock
- For event ordering, we have Lamport's Logical Clock.
- Logical Clock is not based on timing, but on the ordering of events.
- Due to the absence of perfectly synchronized clocks and global time in distributed systems, the order in which two events occur at two different computers cannot be determined based on the local time at which they occur.
- However, under certain conditions, it is possible by analysing the behaviour exhibited by the underlying computation.

## Lamport's logical clocks

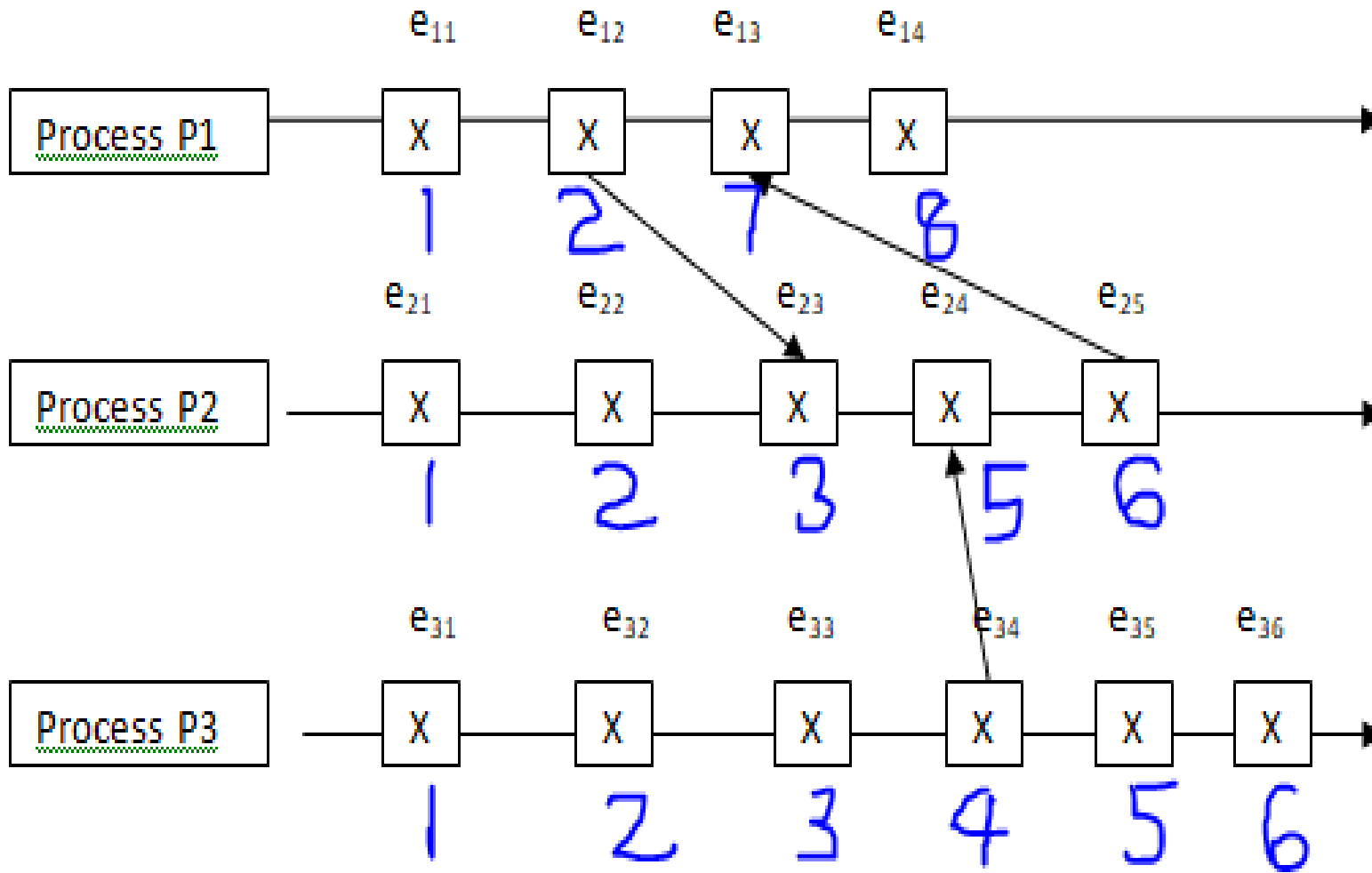
- Is a scheme to order events in a distributed system using logical clocks.
- The execution of events is characterized by a sequence of events.
- Depending on the application, the execution of a procedure could be an event or the execution of an instruction could be one event.
- When processes exchange messages, the execution of a message constitutes one event and receiving a message constitutes another event.



## Lamport's logical clocks

- **Lamport clocks** allow processes to assign sequence numbers (“timestamps”) to messages and other events so that all cooperating processes can agree on the order of related events.
- There is no assumption of a central time source and no concept of *when* events took place.
- Each process maintains a single Lamport timestamp counter.
- Each event in the process is tagged with a value from this counter.
- The counter is incremented before the event timestamp is assigned.
- If a process has four events,  $a$ ,  $b$ ,  $c$ ,  $d$ , the events would get Lamport timestamps of 1, 2, 3, 4, respectively.

# Example



Let's look at an example. The figure shows a bunch of events on three processes. Some of these events represent the sending of a message, others represent the receipt of a message, while others are just local events (e.g., writing some data to a file).

# Implementation of logical clocks n Implementation Rules

- Implementation Rules(guarantee that the logical clocks satisfy the correctness conditions):

[IR1]

Clock  $C_i$  must be incremented between any two successive events in process  $P_i$  :

$$C_i := C_i + 1$$

[IR2]

If event  $a$  is the event of sending a message  $m$  in process  $P_i$  , then message  $m$  is assigned a timestamp  $tm = C_i(a)$  When that same message  $m$  is received by a different process  $P_k$  ,  $C_k$  is set to a value greater than current value of the counter and the timestamp carried by the message

$$C_k := \max(C_k, tm) + 1$$

# Lamport's logical clocks -Definitions

1. HAPPENED BEFORE RELATION( $\rightarrow$ )
2. CASUALLY RELATED EVENTS
3. CONCURRENT EVENTS

# 1. HAPPENED BEFORE RELATION (->)

- The happened before relation captures the casual dependencies between events, i.e., whether two events are casually related or not. The relation  $\rightarrow$  is defined as follows:
  - $a \rightarrow b$ , if  $a$  and  $b$  are events in the same process and  $a$  occurred before  $b$ .
  - $a \rightarrow b$ , if  $a$  is the event of sending a message  $m$  in a process and  $b$  is the event of receipt of the same message  $m$  by another process.
  - If  $a \rightarrow b$  and  $b \rightarrow c$ , then  $a \rightarrow c$ , i.e., " $\rightarrow$ " relation is transitive.
- In distributed systems, processes interact with each other and affect the outcome of events of processes.
- Past events influence future events and this influence among causally related events is referred to as **causal affects**.

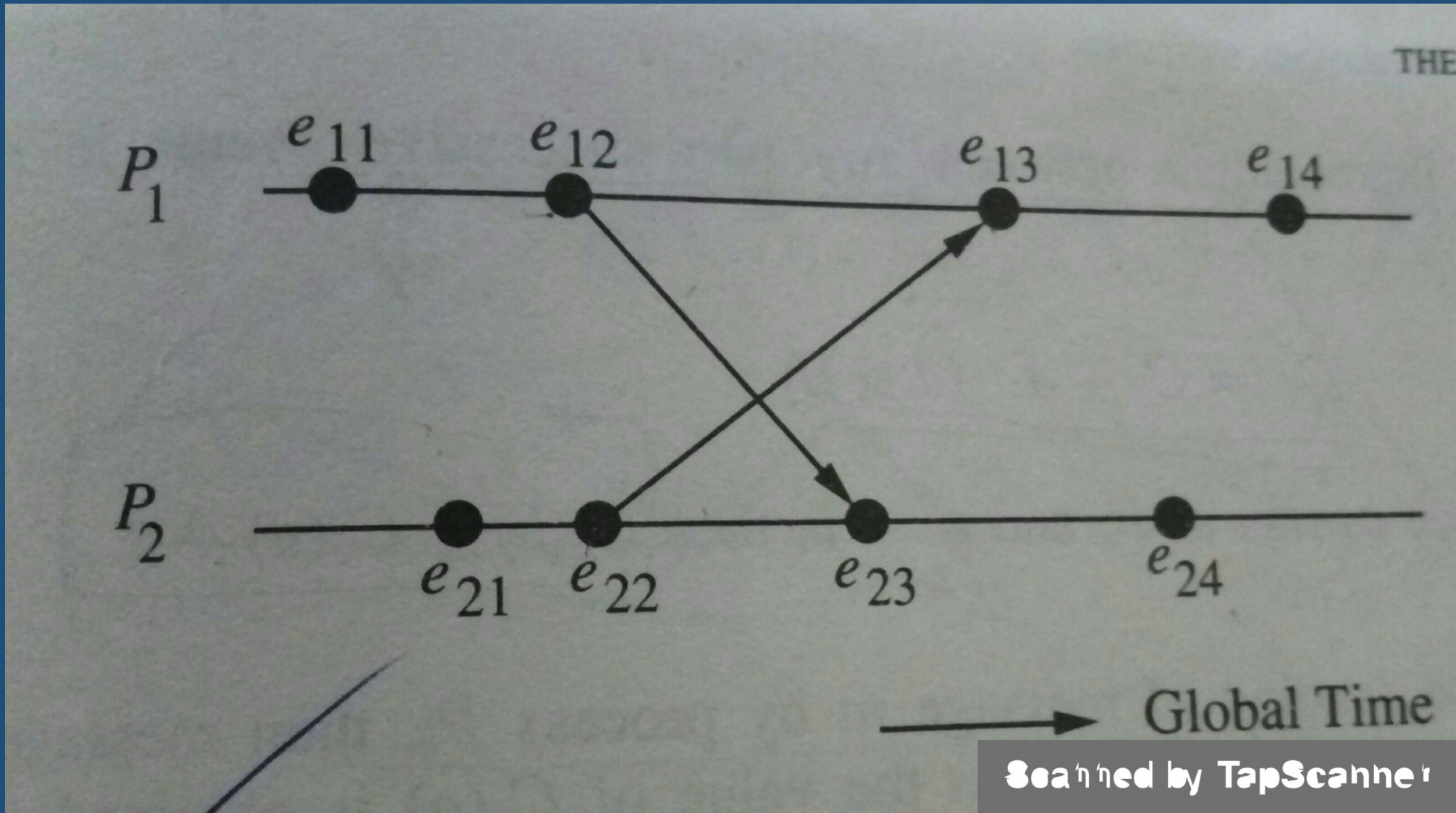
## 2. CAUSALLY RELATED EVENTS

- Event  $a$  causally affects event  $b$  if  $a \rightarrow b$ .

### 3. CONCURRENT EVENTS

- Two distinct events  $a$  and  $b$  are said to be concurrent (denoted by  $a \parallel b$ ) if  $a \not\rightarrow b$  and  $b \not\rightarrow a$ .
- Concurrent events do not affect each other.
- For two events  $a$  and  $b$  in a system, either  $a \rightarrow b$ ,  $b \rightarrow a$ , or  $a \parallel b$ .

# Space time diagram





## Causal ordering of messages

- Deals with the notion of maintaining the same causal relationship that holds among “message send” events with the corresponding “message receive” events
- If  $\text{Send}(M1) \rightarrow \text{Send}(M2)$ , then every recipient of both messages M1 and M2 must receive M1 before M2.
- There are two protocols to make use of vector clocks for the causal ordering.
  - 1. Briman – Schiper – Stephenson Protocol
  - 2. Schiper – Eggli – Sandoz Protocol

# Birman Schiper Stephenson Protocol

- This protocol is used to maintain the causal ordering of the messages.
- i.e. the message which is sent first should be received first.
- If  $\text{send}(M1) \rightarrow \text{send}(M2)$  then for all processes which receive the messages M1 and M2 should receive M1 before M2.

## Features :

- Broadcast based messaging.
- Size of the messages are small.
- More no. of messages are sent.
- Limited state information.

## Key Points :

- Each process increases its vector clock by 1 upon sending of messages.
- Message is delivered to a process if the process has received all the messages preceding to it.
- Else buffer the message.
- Update the vector clock for the process.

# Schiper – Eggli – Sandoz Protocol

- There are three basic principles to this algorithm:
- All messages are time stamped by the sending process.

[Note: This time is separate from the global time talked about in the previous sections. Instead each element of the vector corresponds to the number of messages sent (including this one) to other processes.]

- A message can not be delivered until:
  - All the messages before this one have been delivered locally.
  - All the other messages that have been sent out from the original processs has been accounted as delivered at the receiving process.
- When a message is delivered, the clock is updated.
- This protocol requires that the processes communicate through broadcast messages since this would ensure that only one message could be received at any one time (thus concurrently timestamped messages can be ordered).